WBS TRAINING

ANONYME KLASSEN

OP2 - Die Säulen der Objektorientierten Programmierung

Java

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

WBS TRAINING

Inhalt

1 Anonyme Klassen 2
1.1 Begriff 2
1.2 Aufbau 2
1.3 Zwei Verfahren um anonyme Klassen erstellen 3
1.3.1 Erweitern bestehender Klassen 3
1.3.1.1 Erweitern einer bestehenden nicht-abstrakten Klasse 3
1.3.1.2 Erweitern einer bestehenden abstrakten Klasse 4
1.3.2 Anonyme Klasse, die ein Interface implementiert 5
1.4 Typische Anwendungsfälle 6
1.4.1 GUI Entwicklung – EventHandler 6
1.4.2 Sortieren 6
1.4.3 Methoden 7
1.5 Hinweis zu inneren / lokalen Klassen (nested classes) 8
1.6 UML 9

1 Anonyme Klassen

1.1 Begriff

Anonyme Klassen in Java sind **lokale Klassen ohne Namen**, die du **direkt beim Erstellen eines Objekts definierst**. Sie werden häufig verwendet, wenn du eine **einmalige Implementierung** einer Schnittstelle oder einer abstrakten Klasse brauchst – also dort, wo du eine Klasse nur **ein einziges Mal** benötigst und **nicht erneut wiederverwenden willst**. Mit Java 8 und höher werden sie allerdings oft durch **Lambda-Ausdrücke** (vgl. nächstes Thema) ersetzt, wenn es sich um funktionale Schnittstellen handelt. Werden Klassen umfangreicher und werden diese an verschiedenen Stellen im Code gebraucht, sollte auf anonyme Klassen verzichtet und stattdessen eine benannte Klasse definiert und verwendet werden.

1.2 Aufbau

Anonyme Klassen bestehen aus diesen Elementen:

- `new` Operator
- Namen des Interfaces welches implementiert werden soll oder den Namen der Klasse die erweitert werden soll
- Runde Klammern welche die Argumente für den Konstruktor beinhalten, wie bei normaler Klassenerstellung. Oder leere Klammern für Interfaces
- Geschwungene Klammern für den Körper der anonymen Klasse, in der wir die Logik implementieren, indem wir die Methoden der erweiterten Klasse / Interface überschreiben
- hinter die schließende Klammer ein Semikolon

Nicht möglich mit einer anonymen Klasse:

- einen neuen Konstruktor darin definieren
- auf den Klassennamen zugreifen, da sie keinen hat

1.3 Zwei Verfahren um anonyme Klassen erstellen

1.3.1 Erweitern bestehender Klassen

1.3.1.1 Erweitern einer bestehenden nicht-abstrakten Klasse

Wir erweitern damit die Klasse und können die `gibLaut`-Methode überschreiben. Wir müssen diese aber nicht überschreiben. Hier ist `cat` die Referenz auf die Instanz der anonymen Klasse.

```
public class Katze
{
   public void gibLaut () {System.out.println("Miau!");}
}

Katze cat = new Katze()
{
   @Override
   public void gibLaut()
   {
      System.out.println("Meow!");

   }
};

cat.gibLaut();
```

1.3.1.2 Erweitern einer bestehenden abstrakten Klasse

Wir können abstrakte Klassen und Interfaces nicht direkt instanziieren. Um Objekte davon erzeugen zu können, benötigen wir eine nicht-abstrakte Klasse, welche die abstrakte Klasse erweitert. Das lässt sich als anonyme Klasse umsetzen. Wir müssen die abstrakte Methode überschreiben.

```
public abstract class Lebewesen
{
   public abstract void fressen();
}

Lebewesen loewe = new Lebewesen()
{
   @Override
   public void fressen()
   {
      System.out.println("Fleisch");
   }
};

loewe.fressen();
```

1.3.2 Anonyme Klasse, die ein Interface implementiert

Auch Interfaces kann man nicht direkt instanziieren. Um Objekte davon erzeugen zu können, benötigen wir eine Klasse, welche das Interface implementiert. Das lässt sich ebenso als anonyme Klasse umsetzen. Wir müssen die abstrakten Methoden überschreiben.

```
public interface Fahrbar
{
   public abstract void fahren();
}


Fahrbar pkw = new Fahrbar()
{
   @Override
   public void fahren()
   {
      System.out.println("Pkw Geräusche");
   }
};


pkw.fahren();
```

1.4 Typische Anwendungsfälle

1.4.1 GUI Entwicklung – EventHandler

Hier erschaffen wir uns eine anonyme Klasse vom Typ `EventHandler` bzw. `ActionListener` und überschreiben die `handle` bzw. `actionPerformed` Methode.

```
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button wurde gedrückt!");
    }
});
```

```
btn.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
      System.out.println("Hello World!");
    }
});
```

1.4.2 Sortieren

```
List<String> namen = Arrays.asList("Anna", "Bernd", "Clara");
Collections.sort(namen, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return b.compareTo(a); // absteigend
    }
});
```

1.4.3 Methoden

Aufruf einer Methode, welche ein Objekt einer abstrakten Klasse oder eines Interfaces erwartet. Hier erzeugen wir eine anonyme Klasse direkt als Argument für die Methode.

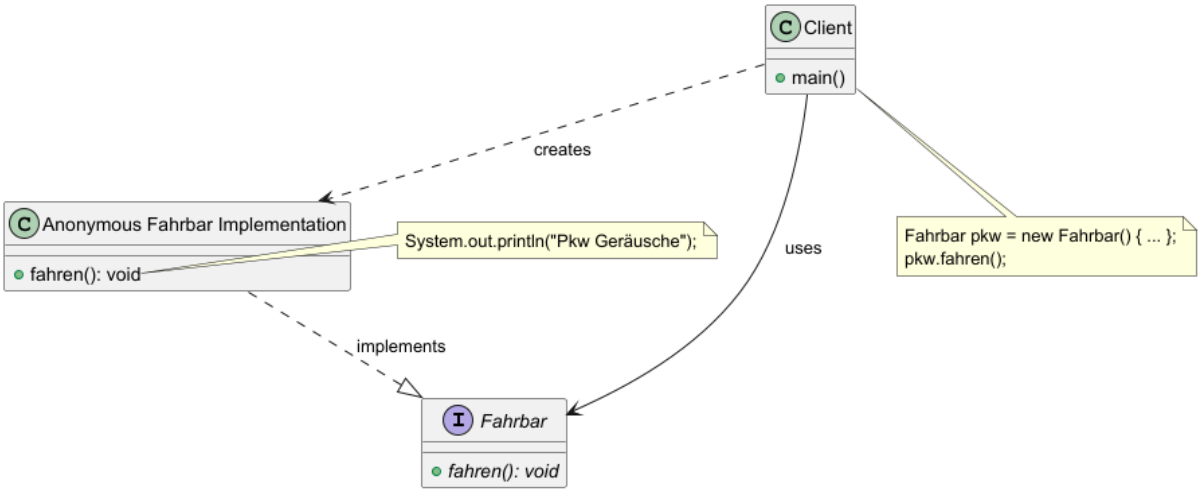
```
public static void ausgabe(Lebewesen lebewesen)
{
   lebewesen.fressen();
}


ausgabe(new Lebewesen()
{
   @Override
   public void fressen ()
   {
      System.out.println("Gras");
   }
});
```

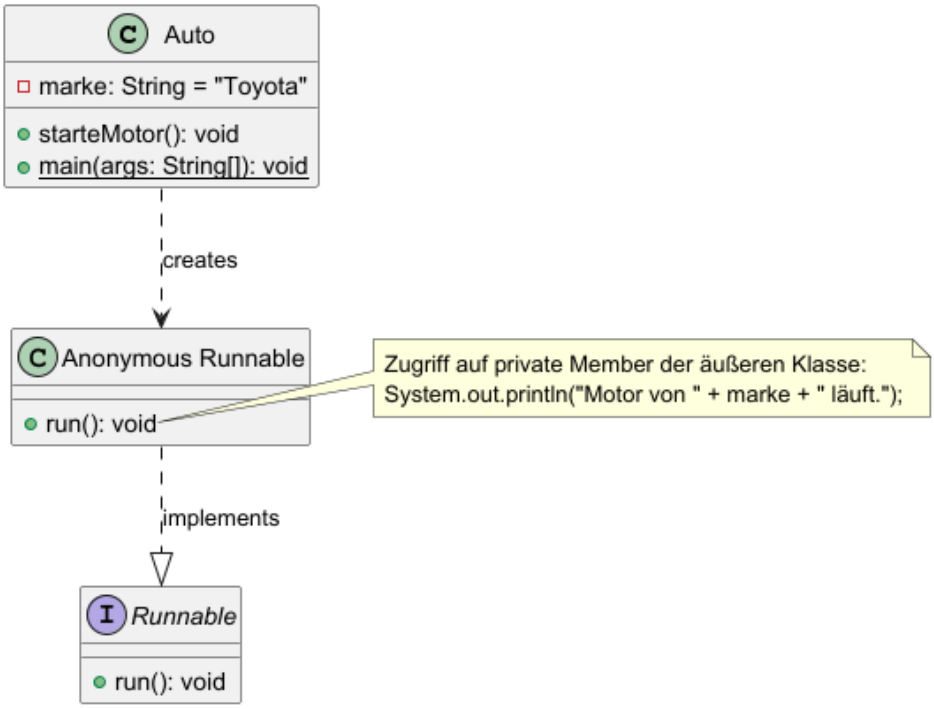
1.5 Hinweis zu inneren / lokalen Klassen (nested classes)

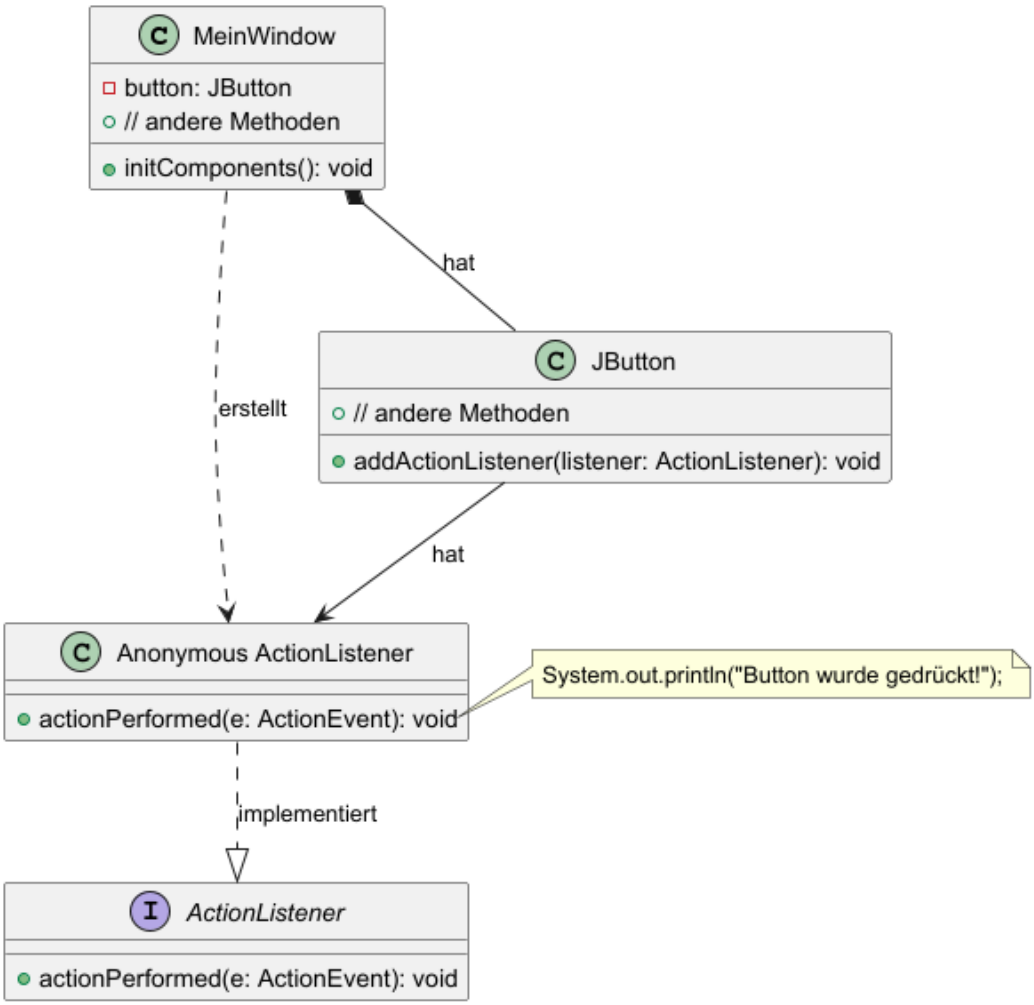
Wenn wir anonyme Klassen erzeugen, erzeugen wir damit innere Klassen. Diese Klassen können auf Member der äußeren Klasse zugreifen.

```
public class Auto {
   private String marke = "Toyota";

   public void starteMotor() {
      System.out.println("Starte Motor...");

      Runnable motor = new Runnable() {
         @Override
         public void run() {
            // Zugriff auf private Member der äußeren Klasse
            System.out.println("Motor von " + marke + " läuft.");
         }
      };
      motor.run();
   }

   public static void main(String[] args) {
      Auto a = new Auto();
      a.starteMotor();
   }
}
```

1.6 UML


Client
main()
creates
Anonymous Fahrbar Implementation
fahren(): void
System.out.println("Pkw Geräusche");
uses
Fahrbar pkw = new Fahrbar() { ... };
pkw.fahren();
implements
Fahrbar
fahren(): void


Auto
marke: String = "Toyota"
starteMotor(): void
main(args: String[]): void
creates
Anonymous Runnable
run(): void
Zugriff auf private Member der äußeren Klasse:
System.out.println("Motor von " + marke + " läuft.");
implements
Runnable
run(): void


MeinWindow
button: JButton
// andere Methoden
initComponents(): void
hat
erstellt
JButton
// andere Methoden
addActionListener(listener: ActionListener): void
hat
Anonymous ActionListener
actionPerformed(e: ActionEvent): void
System.out.println("Button wurde gedrückt!");
implementiert
ActionListener
actionPerformed(e: ActionEvent): void